Traversal Algebra: A Pull–Map–Push Calculus for Parallel Graph Programs

Akira Hayakawa

Research draft
Revision: 2026-07-15 08:47:58 UTC

Abstract

We develop Traversal Algebra, a small algebra for expressing frontier-oriented graph programs independently of a particular GPU execution strategy. A traversal pulls vertex and edge data to an ordered stream of edge contexts, a pure map constructs messages, and monoidal terminals either emit the stream or push messages back to source or destination vertices. This draft fixes the finite ordered multigraph semantics and proves the foundational sequence and fusion laws. The definitions and theorems in this slice are checked in Lean 4, and the executable Lean semantics generates conformance cases for the Massively GPU implementation. We additionally define Monoidal Frontier BSP as an independent source machine and machine-check a forward simulation into the pull–map–push semantic core for one superstep and every finite execution. We then give a signature-relative, callback-free typed syntax and prove that syntax-directed compilation commutes with denotation and preserves every finite execution. For the closed frontier-to-frontier fragment we define an independently compositional Traversal Algebra syntax with structural pulls and arbitrarily nested map/zip trees. We prove that sharing-aware normalization of every such tree into a typed single-push normal form preserves pointwise values, destination inboxes, every finite execution, and empty-frontier termination. Combining normalization with translations to and from typed Monoidal Frontier BSP establishes signature-relative semantic equivalence in both directions for the complete closed expression language. Explicit let bindings avoid the exponential duplication possible with tree substitution. A machine-checked language-level cost semantics proves exact preservation of syntax size, unit work, dependence depth, and local scalar storage, while proving non-increase of full-stream temporary and materialization measures. We also prove exact order and multiplicity semantics for sparse candidate frontiers and give explicit, bidirectionally equivalent observations for emission, source reduction, and destination reduction. A separate typed abstract CubeCL target then derives execution and symbolic workgroup, subgroup, traffic, synchronization, allocation, and materialization costs from the same closed instructions, with exact linear or quasilinear terminal formulas. The Massively/CubeCL implementation is validated separately as an executable artifact and is not an assumption of these language or abstract-machine theorems.

1 Introduction

GPU graph systems commonly organize work around a changing frontier and a bulk-synchronous sequence of parallel edge operations. Gunrock makes the frontier explicit [5]; Ligra separates sparse and dense edge traversals [4]; and PowerGraph exposes a gather–apply–scatter decomposition [2]. GraphBLAS instead specifies graph algorithms using sparse linear algebra over user selected semirings [3]. These systems demonstrate that a small set of composable operations can cover many important algorithms, but coverage by examples alone does not identify the exact expressive boundary.

Traversal Algebra is intended to make that boundary a theorem. Its central object is not a materialized edge list or a particular kernel. It is the denotation of an ordered frontier traversal. Expressions pull source, destination, and edge attributes into an edge context; a map transforms each context; and a terminal emits or reduces the resulting values. Implementations may fuse these stages, change materialization strategies, or select different parallel primitives as long as they preserve the denotation.

The principal result is a characterization theorem, not a collection of examples: for one arbitrary fixed scalar signature, every program in the stated closed, typed, frontier-to-frontier Traversal Algebra grammar has a semantics-equivalent Monoidal Frontier BSP program, and conversely. The proof passes through an independently defined compositional syntax and a sharing-aware single-push normal form. It preserves complete finite runs and empty-frontier termination, has factor-one syntax growth with exact language-level work/depth/local-storage equations, covers dense and stable sparse frontier selection, and separately equates the ordered observations of emission and source/destination reduction. These qualifications identify the proved boundary without calling it unrestricted graph or Turing completeness.

This document has three deliberately separated levels of assurance:

1. mathematical definitions and proofs presented here;
2. kernel-checked counterparts of the language results and a typed abstract CubeCL resource semantics in Lean 4;
3. differential value tests against the public `massively::graph` implementation, plus generated checks of the serialized abstract resource certificates.

The third level is empirical and does not turn the Rust or CubeCL compiler into part of the formal proof. Conversely, the Lean development proves statements about the mathematical language and abstract target without assuming that the emitted CubeCL IR or implementation is correct. Keeping these claims distinct is essential to making later results auditable.

2 Finite ordered multigraphs

Definition 2.1 (Ordered directed multigraph)**.** *An ordered directed multigraph is a tuple*

$$G = (V, E, s, d, \prec)$$

where $V = \{0, \dots, n-1\}$ and $E = \{0, \dots, m-1\}$ are finite, $s, d : E \to V$ give the source and destination of each edge, and $\prec$ totally orders the outgoing edges of every vertex. Parallel edges and self-loops are permitted.

The executable model represents G as an adjacency-row sequence $R_G = [R_0, \dots, R_{n-1}]$. Each R_v is an ordered sequence of destination identifiers. The global edge identifier of the ith item in R_v is

$$\mathsf{base}_G(v) + i, \qquad \mathsf{base}_G(v) = \sum_{u<v} |R_u|.$$

This is precisely the logical information carried by a canonical CSR layout, without making CSR part of the abstract interface.

Definition 2.2 (Frontier)**.** *A frontier is a finite sequence $F \in V^*$. It is not a set. Its order and multiplicity are observable: if v occurs twice, its outgoing row is traversed twice.*

Definition 2.3 (Edge context)**.** *An edge context is a triple*

$$c = (\mathsf{src}(c), \mathsf{dst}(c), \mathsf{eid}(c)) \in V \times V \times E.$$

For $v \in V$, let $X_G(v)$ be the sequence of contexts obtained from R_v in row order. For $F = [v_0, \ldots, v_{k-1}]$, define

$$\mathsf{trav}_G(F) = X_G(v_0) +\!\!+ \cdots +\!\!+ X_G(v_{k-1}),$$

where $+\!\!+$ denotes sequence concatenation.

The regression-oracle model uses natural-number identifiers and assigns no meaning to an invalid graph or to a frontier identifier outside V; its Lean functions remain total for convenient execution. The completeness model uses the type `Fin` n for every vertex reference instead. Valid sources, destinations, and frontier entries are therefore enforced by construction in the simulation theorems rather than carried as side conditions.

3 The pull–map–push calculus

An edge expression denotes a function from contexts to values. The structural fragment contains

$$\mathsf{sourceId}, \quad \mathsf{destinationId}, \quad \mathsf{edgeId}.$$

Attribute expressions extend this fragment by precomposition with s, d, or the edge identifier. Thus a source attribute $a : V \to A$ denotes $a \circ \mathsf{src}$, and similarly for destination and edge attributes. This is the *pull* phase.

For an expression $e : C_G \to A$ and pure function $f : A \to B$, mapping and emission have the denotation

$$\mathsf{emit}_G(F, e, f) = \mathsf{map}(f \circ e, \mathsf{trav}_G(F)).$$

The implementation is not required to construct the intermediate context or expression streams.

Let $M = (A, \oplus, 0)$ be a monoid. Source reduction produces one value for every frontier occurrence:

$$\mathsf{reduceSrc}_G(F, e, f, M)[i] = \bigoplus_{c \in X_G(F[i])} f(e(c)).$$

An isolated vertex therefore contributes 0, and a repeated vertex produces repeated output positions.

Destination reduction pushes values back to a dense vertex domain:

$$\mathsf{reduceDst}_G(F, e, f, M)[v] = \bigoplus_{\substack{c \in \mathsf{trav}_G(F) \\ \mathsf{dst}(c) = v}} f(e(c)).$$

Associativity and identity justify parallel tree reduction. If the lowering may reorder colliding messages, commutativity is additionally required for a deterministic result. Floating-point addition therefore needs either a weaker equivalence relation or a lowering with an explicitly specified order; it is not silently treated as an exact commutative monoid.

4 Foundational laws

Theorem 4.1 (Frontier homomorphism)**.** *For all frontier sequences F_1, F_2,*

$$\mathsf{trav}_G(F_1 +\!+ F_2) = \mathsf{trav}_G(F_1) +\!+ \mathsf{trav}_G(F_2).$$

Proof. By definition, trav_G is the sequence `flatMap` of X_G over its frontier argument. Distributivity of `flatMap` over concatenation gives the result. □

Theorem 4.2 (Emission fusion)**.** *For $p : C_G \to A$ and $q : A \to B$,*

$$\mathsf{map}(q, \mathsf{emit}_G(F, p, \mathsf{id})) = \mathsf{emit}_G(F, q \circ p, \mathsf{id}).$$

Proof. This is the functor composition law for finite sequences: $\mathsf{map}(q, \mathsf{map}(p, x)) = \mathsf{map}(q \circ p, x)$. □

Theorem 4.3 (Source-reduction decomposition)**.** *For frontier sequences F_1, F_2,*

$$\mathsf{reduceSrc}_G(F_1 +\!+ F_2, e, f, M) = \mathsf{reduceSrc}_G(F_1, e, f, M) +\!+ \mathsf{reduceSrc}_G(F_2, e, f, M).$$

Proof. Source reduction maps an independent row fold over every frontier occurrence. The sequence map operation distributes over concatenation. □

These laws are intentionally extensional. They justify fusion and partitioned execution while leaving the choice of scans, segmented reductions, atomics, and materialization to the implementation.

5 Machine-checked and executable artifact

The Lean development [1] mirrors the definitions above. Table 1 records stable declaration names rather than depending only on document numbering.

The proof project contains no admitted theorem. Its executable definitions provide two implementation checks. First, Lean evaluates a named regression corpus containing empty graphs, isolated vertices, parallel edges, self-loops, and repeated frontier entries, then emits the inputs and expected results as Rust source. Second, the same definitions compile to a native oracle accepting arbitrary valid CSR graphs and frontiers. A property-based Rust harness generates bounded inputs, queries this Lean oracle, and compares edge contexts and both reductions with the public Massively GPU operations. Counterexamples are shrunk at the graph and frontier level; no independent Rust traversal semantics is present in the harness.

The persistent oracle also serializes the proved abstract CubeCL resource certificate for scalar queries. Generated tests vary workgroup and subgroup geometry and check graph dimensions, exact fused emission/source/destination formulas, the modeled CSR-control work, atomic counts, and every field of the sequential control-plus-terminal decomposition. These checks validate the protocol boundary; the universal equations are Lean theorems rather than conclusions drawn from the finite campaign.

Passing finitely many generated inputs demonstrates agreement on those inputs; it does not prove universal implementation correspondence. The trusted boundary includes the Lean compiler, a small textual protocol, the Rust test harness, CubeCL, and the device stack. End-to-end verification of that stack is explicitly outside the theorem's scope: the running implementation and property tests establish artifact-level implementability, while Theorem 6.17 concerns the mathematical languages.

6 Equivalence with Monoidal Frontier BSP

Turing completeness is not the primary target. It can often be obtained by encoding a tape in a path graph, but says little about bulk parallel structure, device residency, or asymptotic work. We instead seek a characterization that matches the intended execution model.

Definition 6.1 (Monoidal frontier BSP source machine)**.** *A monoidal frontier bulk-synchronous program operates on a finite ordered multigraph. Let S, P, and A be the vertex-state, edge-payload, and message types. Its semantic operations are an edge message function*

$$\mu : V \times V \times S \times S \times P \to A,$$

a lawful commutative monoid $(A, \oplus, 0)$, a vertex update $u : V \times S \times A \to S$, and a next-frontier selector $q : V^* \times (V \to S) \times (V \to S) \to V^*$. Its first argument is the ordered stream of destination candidates encountered by scanning the active adjacency rows. A barrier configuration is a pair (σ, F) of a dense store and an ordered frontier.

For destination v, the source machine constructs $I_{\sigma,F}(v)$ by a nested left fold: first over sources in F, then over each source adjacency row, combining $\mu(s, d, \sigma(s), \sigma(d), p)$ exactly when $d = v$. It then sets

$$\sigma'(v) = u(v, \sigma(v), I_{\sigma,F}(v)), \qquad F' = q(D_G(F), \sigma, \sigma'),$$

where $D_G(F)$ is built by the same nested frontier/adjacency scan and retains every destination occurrence in exact scan order. This definition does not invoke the flattened traversal or any target operator.

Definition 6.2 (Compiled pull–map–push step)**.** *At state σ, compile the edge map*

$$\widehat{\mu}_\sigma(c) = \mu(\mathsf{src}(c), \mathsf{dst}(c), \sigma(\mathsf{src}(c)), \sigma(\mathsf{dst}(c)), \mathsf{payload}(c)).$$

The target flattens the frontier and adjacency rows into $\mathsf{trav}_G(F)$, maps $\widehat{\mu}_\sigma$, pushes messages by destination using $\oplus$, and uses the same u and q at the barrier. Its candidate stream is the destination projection of the flattened traversal. Pulls are re-instantiated against the new store after every superstep at the semantic level; this does not require runtime kernel compilation.

Lemma 6.3 (Nested-fold flattening)**.** *For every graph, state, frontier, destination, and initial accumulator, folding the flattened traversal gives the same accumulator as the source machine's nested frontier/adjacency fold.*

Proof. Induct on each adjacency row to show that mapping it to edge contexts preserves the destination-filtered fold. Then induct on the frontier and use left-fold composition over sequence concatenation. The Lean declarations are `fold_expand_compile` and `fold_traverse_compile`. □

Theorem 6.4 (One-step forward simulation)**.** *For every finite ordered graph, monoidal frontier BSP program, and barrier configuration, the source superstep and its compiled pull–map–push step produce equal stores and equal ordered frontiers.*

Proof. Lemma 6.3, instantiated with the monoid identity, gives pointwise equality of every destination inbox. The source's nested candidate scan and the target's destination projection are also equal as exact lists by `traversalDestinations_eq_nestedDestinations`. The update and selection functions therefore receive extensionally equal arguments. This is checked by `compile_step_correct` in Lean. □

Corollary 6.5 (Finite-execution forward simulation)**.** *For every $k \in \mathbb{N}$ and initial configuration, k source supersteps equal k compiled target supersteps.*

Proof. Induction on k using Theorem 6.4; checked by `compile_run_correct`. □

Theorem 6.6 (Linear-schedule independence)**.** *Let L be any permutation of $\mathsf{trav}_G(F)$. For every destination v, the destination-filtered fold over L equals the ordered target inbox and the source-machine inbox.*

Proof. Two destination-filtered fold actions commute. If neither or only one context targets v, this is immediate. If both do, associativity moves the common accumulator outside and commutativity exchanges their messages. Induction on the list-permutation derivation then preserves the complete fold. Lean checks the generic result in `foldContextAt_perm`, the target result in `inboxAt_schedule_independent`, and its connection to the source in `inbox_schedule_compile_correct`. □

These results quantify over arbitrary Lean types and total semantic functions; they are not graph enumeration or bounded testing. Exact ordered folds make the present simulation stronger than equality up to a bag, and do not require commutativity inside the forward-simulation proof. Theorem 6.6 uses the monoid laws to permit every linear permutation. A refinement theorem for arbitrary abstract reduction-tree shapes remains separate mathematical work; no concrete GPU execution model is required here.

The semantic simulation deliberately quantifies over arbitrary total Lean functions. This makes it a broad specification envelope, but that envelope is too broad to be a GPU language: an arbitrary selector q may perform any global computation. We therefore refine it with syntax rather than claiming that all functions are compilable.

6.1 Signature-relative typed refinement

Definition 6.7 (Typed scalar IR)**.** *For a fixed signature Σ, object-language types and terms are*

$$\tau ::= \mathsf{Bool} \mid \mathsf{Index}_n \mid b \mid \tau_1 \times \tau_2, \qquad t ::= x \mid \mathsf{literal}(\ell) \mid (t_1, t_2) \mid \mathsf{primitive}(o, t) \mid \mathsf{let}\ x = t_1\ \mathsf{in}\ t_2.$$

Here b, ℓ, and o are symbols declared by Σ. The signature assigns denotations to base types, literal symbols, unary primitive symbols, and reduction-monoid symbols; every declared monoid carries its associative, identity, and commutativity proofs. Multi-argument primitives consume product types, matching the algebra's zip-then-map shape. A let binding evaluates its input once and exposes it to a singleton typed environment; it is the explicit sharing node used by normalization. Terms contain no denotation functions directly: they refer to signature symbols instead.

Variables are intrinsically typed de Bruijn references into heterogeneous environments. The message, update, and selection contexts are respectively

$$\Gamma_m = [\mathsf{Index}, \mathsf{Index}, S, S, P], \quad \Gamma_u = [\mathsf{Index}, S, A], \quad \Gamma_q = [\mathsf{Index}, S, S].$$

Consequently, an ill-typed pull or primitive application cannot be constructed.

Definition 6.8 (Closed typed BSP fragment)**.** *A typed source program consists of a message term $m : \Gamma_m \vdash A$, a declared commutative monoid on A, an update term $u : \Gamma_u \vdash S$, and a closed frontier policy. The policy $\mathsf{dense}(p)$, for $p : \Gamma_q \vdash \mathsf{Bool}$, enumerates every valid vertex in ascending order and compacts those satisfying p; $\mathsf{sparsePreserve}(p)$ instead stably filters the ordered destination candidate*

stream, preserving the multiplicity of every accepted occurrence. Its target program contains the same typed terms under explicit traverse, edge-map, destination-push, vertex-update, and next-frontier fields.

Dense selection remains the canonical $|V|$ baseline. Sparse selection makes candidate generation explicit instead of hiding a global function in the model; its exact order, multiplicity, and cost properties are stated below.

Theorem 6.9 (Compilation commutes with denotation)**.** *For every signature Σ, well-typed source program P, and barrier state σ,*

$$\mathsf{denoteAt}(\mathsf{compile}_{\mathrm{syn}}(P), \sigma) = \mathsf{compile}_{\mathrm{sem}}(\mathsf{denote}(P), \sigma).$$

Proof. Compilation preserves each typed scalar term and changes only graph-control structure. Evaluating the message term in its five-field heterogeneous environment is definitionally the semantic edge map; reduction, update, and frontier fields have the same denotations. Lean checks the equality as `Typed.compile_denoteAt`. □

Corollary 6.10 (Typed finite-run compiler correctness)**.** *For every finite ordered graph, typed source program, initial configuration, and $k \in \mathbb{N}$, interpreting k source supersteps equals interpreting k supersteps of its compiled typed pull–map–push target.*

Proof. Combine Theorem 6.9 with the semantic one-step simulation, then induct on k. The declarations are `Typed.compile_step_correct` and `Typed.compile_run_correct`. The typed schedule result `Typed.inbox_schedule_correct` additionally inherits Theorem 6.6. A concrete natural-number path-mass program and evaluator equations are checked in `TraversalAlgebra.TypedExample`, so the fragment is inhabited. □

6.2 Closed Traversal Algebra fragment

Definition 6.11 (Compositional traversal expressions)**.** *Fix one signature Σ and types S, P, A. The edge-stream expression grammar is*

$$e ::= \mathsf{read}(x) \mid \mathsf{literal}(\ell) \mid \mathsf{map}(e, t) \mid \mathsf{zip}(e_1, e_2).$$

The intrinsically typed variable x selects exactly one of source identifier, destination identifier, source state, destination state, or edge payload. In $\mathsf{map}(e, t)$, if e has element type X, then $t : [X] \vdash Y$ is a unary typed scalar term. Zip constructs a product type. There is no arbitrary edge callback and no pre-normalized message term in this grammar.

A closed frontier-to-frontier expression has the nested operator form

$$Q = \mathsf{compact}_p(\mathsf{updateDst}_u(\mathsf{reduceDst}_M(e))).$$

Here e has element type A, M is a declared commutative monoid on A, $u : \Gamma_u \vdash S$, and the compact policy is either $\mathsf{dense}(p)$ or $\mathsf{sparsePreserve}(p)$ for $p : \Gamma_q \vdash \mathsf{Bool}$. Its denotation is defined directly: recursively evaluate e at every context in $\mathsf{trav}_G(F)$, call the type-safe Traversal Algebra *destination terminal, apply u over the vertex store, and execute the selected compact policy. This operational definition invokes neither BSP nor the normalizer. Lean defines the grammar and semantics as `Typed.TraversalAlgebra.Expression.Traversal`, `Destination`, and `Program.step`.*

This fragment is deliberately closed under the observation of a BSP barrier. The *emit* terminal returns an edge stream, and source reduction returns one item per frontier occurrence; neither is a dense-store transition without an additional output observation. They are therefore not silently conflated with the barrier theorem. Section 6.5 treats them with explicit result constructors and independent BSP observers.

Definition 6.12 (Single-push normal form)**.** *A normal-form* Traversal Algebra *superstep is*

$$\mathsf{trav}_G(F) \longrightarrow^{\text{one typed edge term } m} A^* \longrightarrow^{\mathsf{reduceDst}_M} (V \to A) \longrightarrow^{u} (V \to S) \longrightarrow^{\mathsf{compact}_p} V^*.$$

It is represented by `Typed.TraversalAlgebra.Program`*. This is the four-component normal form used by the earlier BSP correspondence, rather than the definition of arbitrary* Traversal Algebra *syntax.*

Define normalization N recursively on traversal expressions:

$$\begin{aligned} N(\mathsf{read}(x)) &= x, \\ N(\mathsf{literal}(\ell)) &= \mathsf{literal}(\ell), \\ N(\mathsf{map}(e,t)) &= \mathsf{let}\ x = N(e)\ \mathsf{in}\ t, \\ N(\mathsf{zip}(e_1,e_2)) &= (N(e_1), N(e_2)). \end{aligned}$$

The binding is `Typed.Term.share`, represented by the `Term.let1` constructor. Unlike tree substitution, it never copies $N(e)$ when t reads x repeatedly: the input is evaluated once, and every use refers to the bound value. Normalization preserves the destination monoid, vertex update, and frontier policy.

Lemma 6.13 (Pointwise fusion)**.** *For every compositional traversal expression e, store σ, and edge context c,*

$$[\![N(e)]\!](\rho_{\sigma,c}) = [\![e]\!]_\sigma(c).$$

Consequently the complete ordered edge streams before and after normalization are equal.

Proof. Structural induction on e. Read and literal cases are definitional. Zip uses both induction hypotheses. The map case uses the shared-binding equation `Term.evaluate_share`. Lean checks the pointwise and whole-stream results as `Traversal.evaluateAt_normalize` and `Traversal.evaluate_normalize`. □

Theorem 6.14 (Closed-expression normalization)**.** *For every closed compositional* Traversal Algebra *expression Q, graph G, barrier configuration C, and $k \in \mathbb{N}$,*

$$\mathsf{run}_{\mathrm{TAExpr}}(Q,G,k,C) = \mathsf{run}_{\mathrm{TANF}}(N(Q),G,k,C).$$

The equality includes the dense store and ordered frontier.

Proof. Lemma 6.13 makes the mapper functions extensionally equal. Applying the same destination fold gives equality of every inbox; Lean checks this as `Destination.inboxAt_normalize`. The update and compact operations are unchanged, giving `Program.normalize_step_correct`. Induction on k gives `Program.normalize_run_correct`.

Conversely, reification R recursively turns normal-form variables into structural reads, pairs into zip nodes, and primitive applications and shared bindings into map nodes. Introducing an explicit binding means that even $N(R(P)) = P$ need not hold syntactically. Lean instead proves pointwise reification as `Traversal.evaluateAt_ofTerm` and `Traversal.evaluate_normalize_ofTerm`, then lifts it to steps and finite runs as `Program.ofNormalForm_run_correct` and `reify_normalize_run_correct`. Both reification round trips are thus semantic, which is the relevant equality for language equivalence. □

6.3 Sharing-preserving quantitative normalization

The semantic theorem alone would permit an exponentially larger target term. To exclude that failure mode, the cost model counts every expression and term constructor, charges unit work to reads, literals, pair construction, and primitive invocation, permits the two branches of a pair to run in parallel, and counts a conservative peak of live per-edge scalar temporaries. A `let1` node counts its input and body once, evaluates them sequentially, and never copies the bound syntax. At program level, if m is the number of active edge occurrences, work additionally charges one monoid action per edge, one update per vertex, and either $|V|$ dense predicate evaluations or one sparse predicate evaluation per candidate occurrence. The abstract balanced-reduction bound is 0 for no messages and $\lfloor \log_2 m \rfloor + 1$ otherwise.

For comparison with an unfused pointwise schedule, let $M(Q)$ count map/zip materialization passes and let $T_G(Q, C)$ count their full-stream temporary value volume. These are language-level quantities; they do not assert that a particular compiler actually materializes or fuses a stream.

Theorem 6.15 (Quantitative sharing-preserving normalization)**.** *For every closed expression Q, graph G, and configuration C,*

$$\begin{aligned} |N(Q)| &= |Q|, & W_G(N(Q), C) &= W_G(Q, C), \\ D_G(N(Q), C) &= D_G(Q, C), & L(N(Q)) &= L(Q), \\ T_G(N(Q), C) &\le T_G(Q, C), & M(N(Q)) &\le M(Q). \end{aligned}$$

In particular, normalization has syntax-size factor exactly one and cannot hide duplicated scalar work behind denotational equivalence.

Proof. Structural induction proves the four exact traversal equations `Traversal.normalize_nodeCount`, `normalize_work`, `normalize_depth`, and `normalize_scalarTemporaries`; the map case uses the additive definitions of `Term.let1`. Constructor elimination lifts them to `Program.normalize_nodeCount`, `normalize_work`, `normalize_depth`, and `normalize_scalarTemporaries`. A normal form has no mandatory intermediate pointwise edge column, yielding `normalize_fullStreamTemporaryValues_le` and `normalize_materializations_le`. □

Let E encode a typed BSP program as a normal-form destination push and let D decode such a normal form as BSP. These two auxiliary translations expose the already verified nested-fold/flattened-fold connection.

Theorem 6.16 (Normal-form syntactic equivalence)**.** *For every typed BSP program P and normal-form* Traversal Algebra *program Q over the same signature,*

$$D(E(P)) = P, \qquad E(D(Q)) = Q.$$

Consequently E is a bijection between BSP syntax and Traversal Algebra *normal forms.*

Proof. Both equations follow by constructor elimination; no semantic equality is used. Lean checks them as `decode_encode` and `encode_decode` in namespace `Typed.TraversalAlgebra`, with injectivity and surjectivity recorded separately. □

For arbitrary compositional expressions define

$$\mathsf{toBSP}(Q) = D(N(Q)), \qquad \mathsf{ofBSP}(P) = R(E(P)).$$

Because R may introduce map nodes whose normalization adds explicit bindings, the full-language round trips are intentionally semantic rather than syntactic. Lean records the BSP-side finite-run round trip as `Program.toBSP_ofBSP_run_correct`; the normal-form translations E and D themselves retain the exact syntactic bijection above.

Theorem 6.17 (Monoidal Frontier BSP–Traversal Algebra equivalence)**.** *For every fixed signature Σ, finite ordered graph G, initial barrier configuration C, and $k \in \mathbb{N}$,*

$$\mathsf{run}_{\mathrm{BSP}}(P, G, k, C) = \mathsf{run}_{\mathrm{TAExpr}}(\mathsf{ofBSP}(P), G, k, C)$$

for every typed BSP program P, and

$$\mathsf{run}_{\mathrm{TAExpr}}(Q, G, k, C) = \mathsf{run}_{\mathrm{BSP}}(\mathsf{toBSP}(Q), G, k, C)$$

for every closed compositional Traversal Algebra *expression Q. Both equalities include the complete dense store and ordered frontier.*

Proof. For normal forms, the Traversal Algebra destination terminal folds the flattened sequence $\mathsf{trav}_G(F)$. Lemma 6.3 identifies that fold pointwise with the independently defined nested BSP fold; Lean checks the central equation as `encode_inbox_correct` and the finite executions as `encode_run_correct` and `decode_run_correct`. Compose these results with Theorem 6.14. Lean checks the two equations for the full expression grammar as `Program.ofBSP_run_correct` and `Program.toBSP_run_correct`. Their existential forms, `bsp_to_ta_complete` and `ta_to_bsp_complete`, state directly that every program on either side has a semantics-equivalent program on the other. □

Corollary 6.18 (Empty-frontier termination)**.** *Under the conventional driver rule that execution halts when the frontier is empty, a BSP program reaches a halting configuration if and only if its compositional* Traversal Algebra *embedding does; an arbitrary closed* Traversal Algebra *expression reaches one if and only if its normalized BSP translation does.*

Proof. Theorem 6.17 gives equality of the ordered frontier at every step index, so existence of an index with the empty frontier is preserved. Lean checks both directions as `ofBSP_terminates_iff` and `toBSP_terminates_iff`. □

6.4 Sparse frontier order, multiplicity, and work

Let $D_G^{\mathrm{nested}}(F)$ be the candidate stream produced directly by the BSP source machine and let $D_G^{\mathrm{flat}}(F)$ project destinations from $\mathsf{trav}_G(F)$. Both are lists rather than sets: parallel edges, repeated frontier entries, and repeated destinations remain observable.

Theorem 6.19 (Sparse candidate preservation)**.** *For every graph and ordered frontier,*

$$D_G^{\mathrm{flat}}(F) = D_G^{\mathrm{nested}}(F)$$

as exact lists. For every typed predicate p, $\mathsf{sparsePreserve}(p)$ is a stable filter of this list. It introduces no occurrence; every accepted vertex retains exactly its candidate multiplicity, and every rejected vertex has multiplicity zero. If the candidate count is c, its predicate work is exactly $cW(p)$; hence it is no greater than dense selection's $|V|W(p)$ whenever $c \le |V|$.

Proof. Induct on the frontier. In the cons case, both constructions append the destination projection of the active source row to their recursive tail; the head projections are definitionally equal and the induction hypothesis closes the tail. Stable-filter order follows from the standard list-filter sublist lemma. For a fixed vertex, case analysis on the Boolean predicate gives the original count in the accepted case and zero in the rejected case. Expanding the two work definitions gives $cW(p)$ and $|V|W(p)$, so monotonicity of multiplication proves the conditional inequality. Candidate duplicates can make $c > |V|$, and the theorem deliberately does not erase that cost. The Lean declarations are mapped in Table 1. □

Because both machines pass these equal candidate lists to the same frontier policy, the step, finite-run, and equivalence theorems above cover both dense and sparse policies without weakening ordered-frontier equality.

6.5 Terminal observations

Barrier configurations are not the right codomain for every terminal. We therefore use the disjoint observation type

$$\mathsf{Result}(A) = \mathsf{emitted}(A^*) + \mathsf{sourceReduced}(A^*) + \mathsf{destinationReduced}(V \to A).$$

The distinct list constructors make the indexing contract explicit: emission contains one value per traversed edge occurrence in traversal order, whereas source reduction contains one value per frontier occurrence after folding its adjacency row in order. Destination reduction observes a dense store.

The TA observer evaluates an arbitrary compositional traversal expression and invokes the corresponding public terminal. Independently, the BSP observer uses a typed edge term and nested frontier/adjacency recursion; it does not call flattened traversal for its definition.

Theorem 6.20 (Bidirectional terminal-observation equivalence)**.** *Every TA emission, source-reduction, or destination-reduction observer has a normalized BSP observer with exactly the same* Result*. Conversely, every typed BSP observer has a reified TA observer with exactly the same result. The equalities preserve list order and all occurrence multiplicities. Under the unit-cost model, TA-to-BSP normalization also preserves observation work exactly for all three terminals.*

Proof. Row induction proves ordered emission and source folds; frontier induction proves their concatenation and every nested destination fold. The generic lemmas are `Observation.Proof.map_traverse`, `source_correct`, and `destination_correct`. Pointwise normalization and reification then give `Terminal.toBSP_observe_correct` and `BSP.Terminal.toTA_observe_correct`. Exact work preservation is `Observation.Cost.toBSP_work`. □

Thus semantic expressive completeness and the converse hold for the entire stated closed expression grammar, not merely for a record chosen to be isomorphic to BSP. This result is relative to one arbitrary but fixed signature Σ; it does not require translating Σ to an execution platform or proving an implementation.

6.6 Orthogonal signature transport

Signature transport is useful when changing scalar vocabularies, but is not a premise of Theorem 6.17.

Definition 6.21 (Signature lowering)**.** *A lowering $L : \Sigma \Rightarrow \Sigma'$ maps every literal, primitive, and monoid symbol to a symbol of the same object-language type in Σ'. It also carries the denotational equations*

$$[\![L(\ell)]\!]_{\Sigma'} = [\![\ell]\!]_{\Sigma}, \qquad [\![L(o)]\!]_{\Sigma'}(x) = [\![o]\!]_{\Sigma}(x), \qquad [\![L(M)]\!]_{\Sigma'} = [\![M]\!]_{\Sigma}.$$

The last equality is equality of the complete reduction operation, including its identity and combine function. Lowerings have an identity and compose by composing symbol maps and their equations.

Lowering extends structurally to terms, dense and sparse frontier policies, and both source and target programs. Variables, products, and let-binding structure are unchanged.

Lemma 6.22 (Term-lowering preservation)**.** *For every well-typed term $\Gamma \vdash_\Sigma t : \tau$, environment ρ, and certified lowering $L : \Sigma \Rightarrow \Sigma'$,*

$$[\![\mathsf{lower}_L(t)]\!]_{\Sigma'}\rho = [\![t]\!]_\Sigma\rho.$$

Proof. Structural induction on t. Variables are unchanged, literal and primitive cases use the corresponding certificate equations, the product case uses two induction hypotheses, and the let case preserves both the bound input and its singleton body environment. Lean checks this as `Typed.Term.evaluate_lower`. It also checks identity and composition for terms, policies, and complete programs. □

Theorem 6.23 (Compilation/lowering naturality)**.** *For every typed source program P and certified lowering L,*

$$\mathsf{compile}_{\mathrm{syn}}(\mathsf{lower}_L(P)) = \mathsf{lower}_L(\mathsf{compile}_{\mathrm{syn}}(P)).$$

Moreover, for every finite graph, initial configuration, and $k \in \mathbb{N}$, the lowered source and the lowered compiled target produce equal configurations after k supersteps.

Proof. The syntactic square is fieldwise definitional equality and is checked by `Typed.compile_lower`. Lemma 6.22 gives equal source and target denotations before and after lowering; combining it with Corollary 6.10 gives the finite-run square checked by `Typed.lower_compile_run_correct`. □

No concrete execution platform occurs in these statements. The non-identity example in `TraversalAlgebra.TypedExample` merely witnesses transport between two mathematical symbol datatypes. It is not implementation verification and is not needed by the BSP–Traversal Algebra equivalence.

6.7 Abstract CubeCL resource refinement

The platform-independent equivalence above is complemented, rather than replaced, by a typed abstract CubeCL target. Its machine parameters are a one-dimensional workgroup size and subgroup size. A cost vector records logical and padded work-items, scheduled subgroups, scalar work, critical-path span, global loads and stores, host-visible reads, atomic operations, barriers, launches, allocation volume, and full-stream materializations. Sequential kernel plans compose these vectors fieldwise.

The target program is a closed instruction datatype for emission, source reduction, destination sort/reduce, destination atomic reduction, and a materialized CSR-control prefix. It is not a record in which arbitrary semantic code can be paired with an unrelated inexpensive plan: execution and resource interpretation are both determined by the same instruction. An atomic instruction additionally carries an action, a proof that the action is the declared monoid combination, and its scalar atomic-operation count under a chosen storage-width interpretation.

Theorem 6.24 (Abstract CubeCL terminal resources)**.** *Let m be active-edge occurrences, f frontier occurrences, v vertices, w normalized expression work, q output storage words, ℓ global-load words per edge, and $r(m) = 0$ for $m = 0$ and $\lfloor \log_2 m \rfloor + 1$ otherwise. The typed lowering preserves the exact terminal observation. Its fused emission has work $m(w + q)$, loads $m\ell$, stores mq, and no*

expression-stream materialization. Source reduction has work $f + m(w + 1)$. Given atomic evidence with a scalar atomics per combine, destination reduction has work $v + m(w + 1)$ and exactly ma atomics. The general destination path has work

$$v + mw + mr(m) + m.$$

For the modeled current CSR-control prefix, work is bounded by

$$|E| + v + 6f + 11m + 3,$$

where the whole-topology term $|E|$ remains explicit. Prefixing this control plan changes cost by exact sequential composition and does not change the observation.

Proof. Semantic refinement is `CubeCL.lower_correct`; atomic action refinement is `Program.execute_destinationAtomic`. The exact terminal equations are `lowerEmit_scalarWork`, `lowerEmit_globalLoads`, `lowerEmit_globalStores`, `lowerSource_scalarWork`, `lowerDestinationAtomic_scalarWork`, `lowerDestinationAtomic_operations`, and `lowerDestinationSort_scalarWork`. The CSR bound and composition are `materializedCsrControl_linearWork` and `Program.cost_withMaterializedCsrControl`. □

These are exact equations in the stated symbolic machine, not elapsed-time predictions. The scan primitive is represented by a high-level hierarchical resource contract. Cache and coalescing effects, occupancy limits, atomic contention latency, and peak allocation liveness are deliberately not assigned hardware weights, and no theorem currently derives this certificate from emitted CubeCL IR.

The proved boundary is now semantic and quantitative at both the language and abstract-target levels, including all three terminal observations and both frontier policies. The remaining refinement programme is narrower:

1. refine arbitrary parallel reduction-tree shapes to the permutation-invariant denotation already proved;
2. prove refinement from emitted CubeCL kernel traces to the abstract target, then calibrate backend-weighted latency, cache/coalescing, contention, transfer, and peak-residency bounds;
3. derive representative algorithms, including the Gunrock suite, as coverage evidence rather than as the completeness proof itself.

The executable Massively/CubeCL realization and its property tests remain artifact evidence for implementability, not an assumption of these formal language results. A Turing-machine simulation or a connection to fixed-point logics could still be added as a secondary expressiveness result, but it would say less about frontier-parallel structure than the correspondence proved here.

References

[1] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021.

[2] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In *10th USENIX Symposium on Operating Systems Design and Implementation*, pages 17–30, 2012.

[3] Jeremy Kepner, Peter Aaltonen, David Bader, Aydin Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, Scott McMillan, Carl Yang, and John D. Owens. Mathematical foundations of the graphblas. *2016 IEEE High Performance Extreme Computing Conference*, 2016.

[4] Julian Shun and Guy E. Blelloch. Ligra: A lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 135–146, 2013.

[5] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2016.

Paper object	Lean declaration
Ordered multigraph	`TraversalAlgebra.Graph`
Edge context	`TraversalAlgebra.EdgeContext`
trav_G	`TraversalAlgebra.Graph.traverse`
Emission	`TraversalAlgebra.emit`
Source reduction	`TraversalAlgebra.reduceBySource`
Destination reduction	`TraversalAlgebra.reduceByDestination`
Theorem 4.1	`traverse_append`
Theorem 4.2	`emit_map_fusion`
Type-safe completeness graph	`Verified.OrderedGraph`
Type-safe TRAVERSAL ALGEBRA terminals	`Verified.TraversalAlgebra.emit`, `reduceBySource`, `reduceByDestination`
Semantic BSP source	`Verified.MonoidalFrontierBSP.Program`
Semantic pull–map–push target	`Verified.PullMapPush.Plan`
Schedule independence	`Verified.PullMapPush.inboxAt_schedule_independent`
Theorem 6.4	`Verified.compile_step_correct`
Corollary 6.5	`Verified.compile_run_correct`
Typed values and sharing terms	`Typed.ValueType`, `Typed.Term.let1`, `Typed.Term.share`
Typed BSP source	`Typed.MonoidalFrontierBSP.Program`
Typed pull–map–push target	`Typed.PullMapPush.Program`
Theorem 6.9	`Typed.compile_denoteAt`
Corollary 6.10	`Typed.compile_run_correct`
Compositional TRAVERSAL ALGEBRA grammar	`Typed.TraversalAlgebra.Expression.Traversal`, `Program`
Lemma 6.13	`Traversal.evaluateAt_normalize`, `evaluate_normalize`
Theorem 6.14	`Program.normalize_run_correct`
Theorem 6.15	`Expression.Program.normalize_nodeCount`, `normalize_work`, `normalize_depth`, `normalize_scalarTemporaries`
Single-push TRAVERSAL ALGEBRA normal form	`Typed.TraversalAlgebra.Program`
Nested BSP fold equals TA terminal	`encode_inbox_correct`
Theorem 6.16 (normal form)	`decode_encode`, `encode_decode`
Theorem 6.17	`Program.ofBSP_run_correct`, `toBSP_run_correct`
Empty-frontier termination	`Program.ofBSP_terminates_iff`, `toBSP_terminates_iff`
Theorem 6.19	`OrderedGraph.traversalDestinations_eq_nestedDestinations`, `FrontierPolicy.sparsePreserve_count`
Theorem 6.20	`Observation.Terminal.toBSP_observe_correct`, `Observation.BSP.Terminal.toTA_observe_correct`
Certified signature translation	`Typed.Signature.Lowering`
Lemma 6.22	`Typed.Term.evaluate_lower`
Theorem 6.23	`Typed.compile_lower`, `Typed.lower_compile_run_correct`
Abstract CubeCL target and cost vector	`Typed.CubeCL.Program`, `Typed.CubeCL.Cost`
Theorem 6.24	`Typed.CubeCL.lower_correct`, `lowerEmit_scalarWork`, `lowerDestinationSort_scalarWork`, `materializedCsrControl_linearWork`

Table 1: Paper-to-Lean declaration map. Rows beginning "Typed" are nested below "Verified"; expression-program names are in `Typed.TraversalAlgebra.Expression`, while normal-form names are in `Typed.TraversalAlgebra`; observer names continue below its `Observation` namespace. All names omit the common top-level namespace.